

Monitoreo de temperatura-humedad-ACC

Juan Jose Arteaga
Taylor Daniel Romero Moreno
Electrónica digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	4
6. Explicación de funciones principales	4
7. Interfaz de usuario (si aplica)	4
8. Procedimiento de prueba	5
9. Manejo de errores y seguridad	5

1. Identificación del proyecto

- **Nombre del proyecto:** Monitoreo en tiempo real con ESP32 y Micropython
- **Integrantes del equipo:** Juan Jose Arteaga, Taylor Daniel Romero Moreno

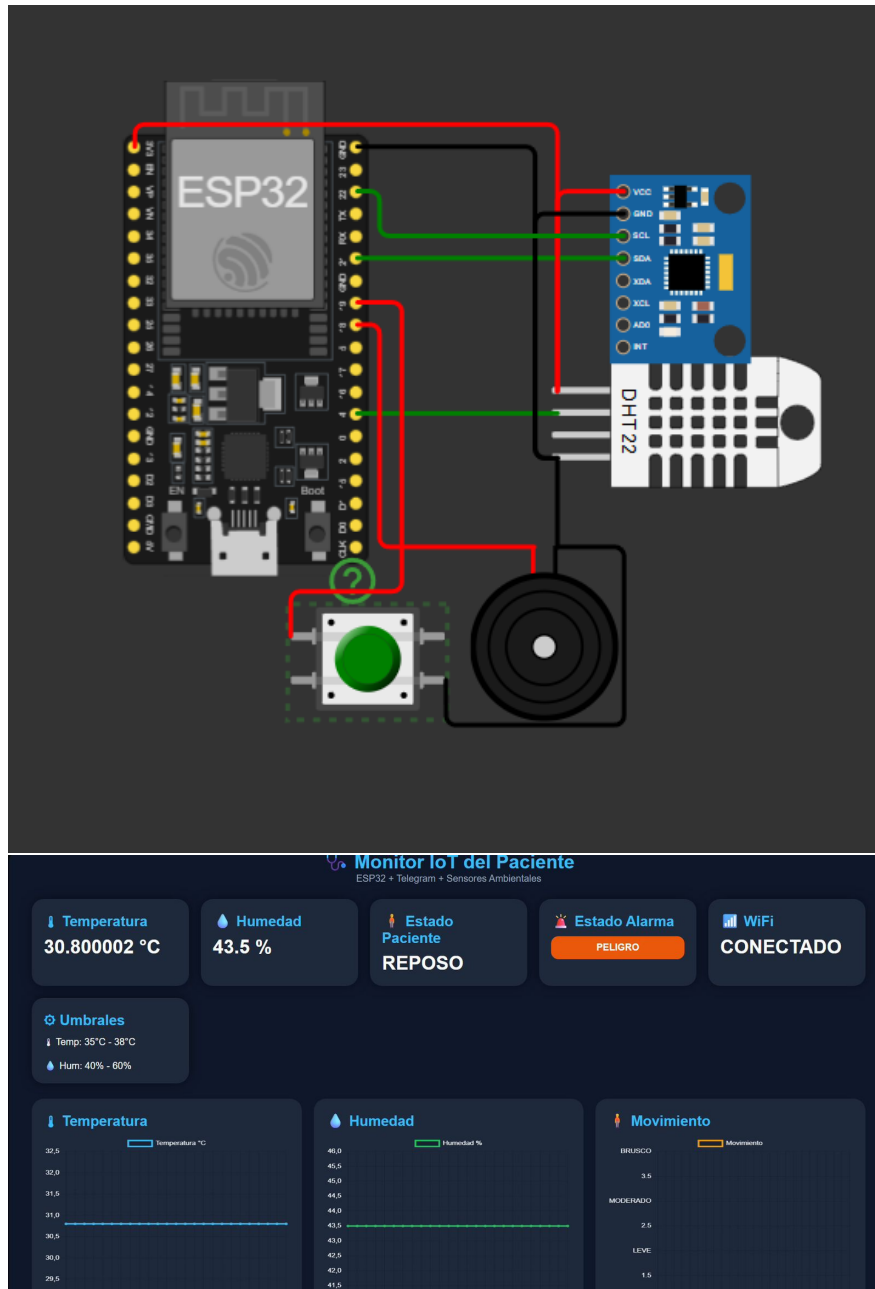
2. Resumen

Este proyecto consiste en el diseño de un sistema de monitoreo en tiempo real con posibilidades de visualización de datos en software extra e implementación de controles y herramientas externas como telegram

3. Descripción del hardware

- **ESP32** (modelo de 30 pines)
- **DHT** Pin 4
- **Boton panico** Pin 19
- **SDA** Pin 21
- **SCL:** Pin 22
- **Buzzer** Pin 18

Diagrama de conexión:



4. Descripción del software

- **Lenguaje usado:** MicroPython
- **Librerías:** machine, network, dht, time, math, urequests, structs, gc
- **IDE:** Thonny

Flujo general del programa:

1. Importacion de librerias

2. Configuración de conexiones, servidores y hardware
3. Lectura de los sensores
4. Inicio de While True principal
5. Acciones de los sensores a la página web

5. Estructura del código

- Archivo principal: `main.py`
- Se usa página web y telegram.
- Arquitectura lineal con ciclos infinitos con lectura y cambios en lectura en tiempo real.

6. Explicación de funciones principales

- `wifi.connect(WIFI_SSID, WIFI_PASSWORD)`: Fundamental para conectar el ESP32 a internet y permitir el envío de datos y alertas.
- `sensor.measure()`: Realiza la lectura del sensor DHT22 para obtener temperatura y humedad.
- `magnitude = math.sqrt(ax**2 + ay**2 + az**2)`: Calcula la magnitud total del movimiento usando los 3 ejes del acelerómetro.
- `if temperature > TEMP_MAX::` Detecta si la temperatura supera el límite establecido.
- `if panic_button.value() == 0::` Detecta cuando el usuario presiona el botón de pánico.
- `send_to_server(payload)`: Envía toda la información al servidor Flask para actualizar el dashboard.
- `movement, movement_level = detect_movement()`: Esto interpretará el nivel de movimiento del sistema según la intensidad en los ejes
- `alarm = CRITICO`: Detecta la pulsación del botón en el hardware cuando se detecta una emergencia
- `payload = { ... }`: Aquí se crea la estructura principal de datos que contiene toda la información del sistema.

7. Interfaz de usuario (si aplica)

Visualización de datos en tiempo real en servidor aparte e incluso interacciones mediante telegram

8. Procedimiento de prueba

1. Cargar el código en el ESP32.
2. Conectar el ESP32 a energia.
3. Conectar todo a la misma red wi fi
4. Abrir servidor
5. Comienzo de lectura de sensores y visualizacion en tiempo real

9. Manejo de errores y seguridad

- Limites de lectura previamente establecidos para garantizar correcto funcionamiento